

Weaponizing Words: Direct & Indirect Prompt Injection Attacks on LLM

Pavan Reddy*

George Washington University

Washington, DC, USA

pavan.reddy@gwmail.gwu.edu

Abstract

As AI systems—particularly LLM applications—proliferate across products and workflows, security has not kept pace. Prompt injection is the failure mode most likely to turn well-intentioned LLM applications into unsafe systems [4]. This 75-minute, code-first tutorial focuses exclusively on the two forms that matter in practice: direct injection through trusted input channels and indirect injection via retrieved or linked content. Using two reproducible demos—*EmailGPT* (direct) and *EchoLeak* (indirect)—participants will reproduce end-to-end attacks that override system instructions, exfiltrate secrets, and trigger unauthorized actions, then harden the pipelines step by step. Labs run on free notebooks with small open models and mock tools; no API keys required. We distill why guardrails and “better prompts” alone are insufficient—instruction-following bias, collapsed trust boundaries, and tool autopilot—and build a deployable defense-in-depth stack: input canonicalization, instruction/data separation, untrusted-context labeling, structured I/O contracts, provenance and taint tracking, deny-by-default tool policies, and red-team regression tests. Attendees leave with working intuition, tested code, and a practical checklist they can drop into RAG flows, agents, and email assistants the same day.

ACM Reference Format:

Pavan Reddy. 2025. Weaponizing Words: Direct & Indirect Prompt Injection Attacks on LLM. In *26th Annual ACM Conference on Cybersecurity and Information Technology Education (ACM SIGCITE 2025)*, November 06–08, 2025, Sacramento, CA, USA. ACM, New York, NY, USA, 2 pages. <https://doi.org/10.1145/3769694.3771165>

Introduction

This tutorial directly supports Embracing Our exCITing Identity by giving CS/IT educators deployable defenses for GenAI systems used in courses and operations. Large Language Models (LLMs) are now embedded in email assistants, retrieval-augmented generation (RAG) systems, and agentic workflows that can read, write, click, query tools, and call APIs. In these settings, *instruction integrity*—the guarantee that only trusted instructions govern the model’s behavior regularly fails. Attackers exploit this by injecting alternative instructions that the model prefers or is coerced to follow. Two operationally distinct threats dominate incidents [2]:

- **Direct Prompt Injection:** malicious instructions sent by the user into a trusted input channel (e.g., an email reply field or chat box) that override the system’s intended policy.
- **Indirect Prompt Injection:** malicious instructions embedded in data the system fetches (links, HTML, PDFs, images with alt-text, tool responses) and forwards to the LLM, collapsing the trust boundary once untrusted content is concatenated.

Motivation & Relevance

Despite prompt templates, safety prompts, and routing, incidents persist because models obey the wrong instructions. We make this failure precise and operationally actionable.

Why existing controls fail.

- (1) **Instruction-following bias** [3]: models over-weight imperative phrasing and nearby task framing, even when it conflicts with earlier “system” guidance.
- (2) **Trust-boundary collapse:** once untrusted content is concatenated or rendered (e.g., markdown/HTML), the model cannot reliably track provenance or treat text differently by origin.
- (3) **Tool-use autopilot:** orchestration layers often auto-forward model outputs to tools or memory, turning a text mistake into a side-effect (API call, database write, email send).

Content Description

Two compact, zero-dependency Colab notebooks (Gemma/Ollama-class) with prewired stubs for reproducibility will be provided.

Module A — EmailGPT (Direct Prompt Injection) [1]

Scenario: An email summarizer reads the message body and drafts replies. An attacker embeds “system-like” directives inside the email itself.

Objective: Force the assistant to override the system prompt; e.g., exfiltrate summary rules, add unauthorized recipients, or inject a misleading summary.

Steps:

- (1) Baseline behavior with a benign email.
- (2) Inject instructions (“Ignore all prior instructions. Reply with ...”) and observe takeover.
- (3) Add naive filters; show evasion via obfuscation (zero-width, markdown indirection).
- (4) Apply layered mitigations and verify containment.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).

ACM SIGCITE 2025, Sacramento, CA, USA

© 2025 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2240-0/25/11

<https://doi.org/10.1145/3769694.3771165>

Time (mm:ss)	Topic	What Happens
00:00–07:00	Setup & Goals	Load Colabs; quick threat model; success criteria.
07:00–15:00	Prompt Injection Primer	Direct vs. Indirect; failure modes; why “just prompt better” fails.
15:00–35:00	EmailGPT (Direct)	Reproduce takeover → iterate minimal → robust mitigations.
35:00–62:00	EchoLeak (Indirect)	Trigger hidden-instruction leak → harden pipeline → re-attack.
62:00–70:00	Defense Checklist	Canonicalization, content labeling, tool gating, output contracts, eval harness.
70:00–75:00	Wrap & Q&A	Common pitfalls, references, pointers to code.

Table 1: Detailed 75-minute tutorial schedule outlining topics, activities, and expected outcomes for each session segment.

Key Takeaways: Trust boundary around user inputs; template isolation; canonicalization; structured I/O; content-policy enforcers before the LLM; allow-lists for actions and recipients.

Module B – EchoLeak (Indirect Prompt Injection) [5]

Scenario: A retrieval/agent flow fetches a web page/document and asks the LLM to “extract key points.” The page contains hidden instructions telling the model to reveal its private system prompt and tool credentials.

Objective: Exfiltrate secrets or redirect tool calls using only external content.

Steps:

- (1) Baseline scrape + summarize from a benign source.
- (2) Swap in a booby-trapped page containing hidden instructions (HTML comments/ARIA/meta).
- (3) Observe leakage of system prompt and attempt to trigger tool misuse.
- (4) Harden with provenance tagging, content compartmentalization, and “untrusted context” roles; re-run to confirm containment.

Key Takeaways: Treat retrieved content as untrusted code; segregate instructions from data; sign and label system messages; implement cross-context instruction firewalls and deny-by-default tool policies.

Tutorial Format

- Hands-on first. Short intros, then run code.
- Prebuilt infra. Notebooks include mock tools/APIs; no cloud keys required.
- Lightweight models. Runs on free Colab/Ollama-class CPU/GPU; deterministic seeds.
- Deliverables. Two notebooks, one mitigation checklist, one red-team harness snippet.

Audience & Prerequisites

- **Who:** ML/AI engineers, app/security practitioners, students.
- **Prerequisites:** Intermediate Python + LLM familiarity; no security background required.
- **Class size:** Ideal 10–15; scales to 50–80 via pair programming; BYO-laptop; browser-only (Colab/Ollama); overflow welcome as view-only.

Materials Provided

- Printable 1-page checklist
- Links to notebooks and Github Repo.
- Offline zip fallback provided if Colab quota is constrained.

Reproducibility

- 100% open notebooks, permissive licenses, no proprietary APIs.
- Fixed seeds and recorded outputs; synthetic corpora for EchoLeak.
- Self-contained harness to regression-test mitigations after you edit prompts/templates.

Learning Outcomes

- Execute and explain how direct and indirect injections take over LLM behavior.
- Implement practical controls that survive obfuscation and hidden-instruction tricks.
- Integrate an evaluation harness to continuously test for regressions.

Conclusion

Direct and indirect prompt injections remain the clearest path to real incidents. By reproducing EmailGPT and EchoLeak end-to-end and then hardening them, attendees leave with working intuition and a deployable guardrail playbook they can drop into any LLM app the same day.

References

- [1] 2025. EmailGPT and EchoLeak Tutorial Materials. GitHub repository; link provided during session. Supplementary notebooks and code for the tutorial.
- [2] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. 2023. Not What You’ve Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec ’23)* (Copenhagen, Denmark). Association for Computing Machinery, New York, NY, USA, 79–90. doi:10.1145/3605764.3623985
- [3] Raz Lapid, Ron Langberg, and Moshe Sipper. 2024. Open Sesame! Universal Black-Box Jailbreaking of Large Language Models. In *ICLR 2024 Workshop on Secure and Trustworthy Large Language Models (SeT-LLM)*. doi:10.48550/arXiv.2309.01446 arXiv:2309.01446 [cs.CL]
- [4] OWASP Foundation. 2025. OWASP Top 10 for Large Language Model Applications: 2025. <https://owasp.org/www-project-top-10-for-large-language-model-applications/>. LLM01: Prompt Injection.
- [5] Pavan Reddy and Aditya Sanjay Gujral. 2025. EchoLeak: The First Real-World Zero-Click Prompt Injection Exploit in a Production LLM System. arXiv:2509.10540 [cs.CR] <https://arxiv.org/abs/2509.10540>